

Ошибки и исключения в Python



ДАНИЛА ЕВГРАФОВ

**Chief Technology Officer,
Zvonobot**

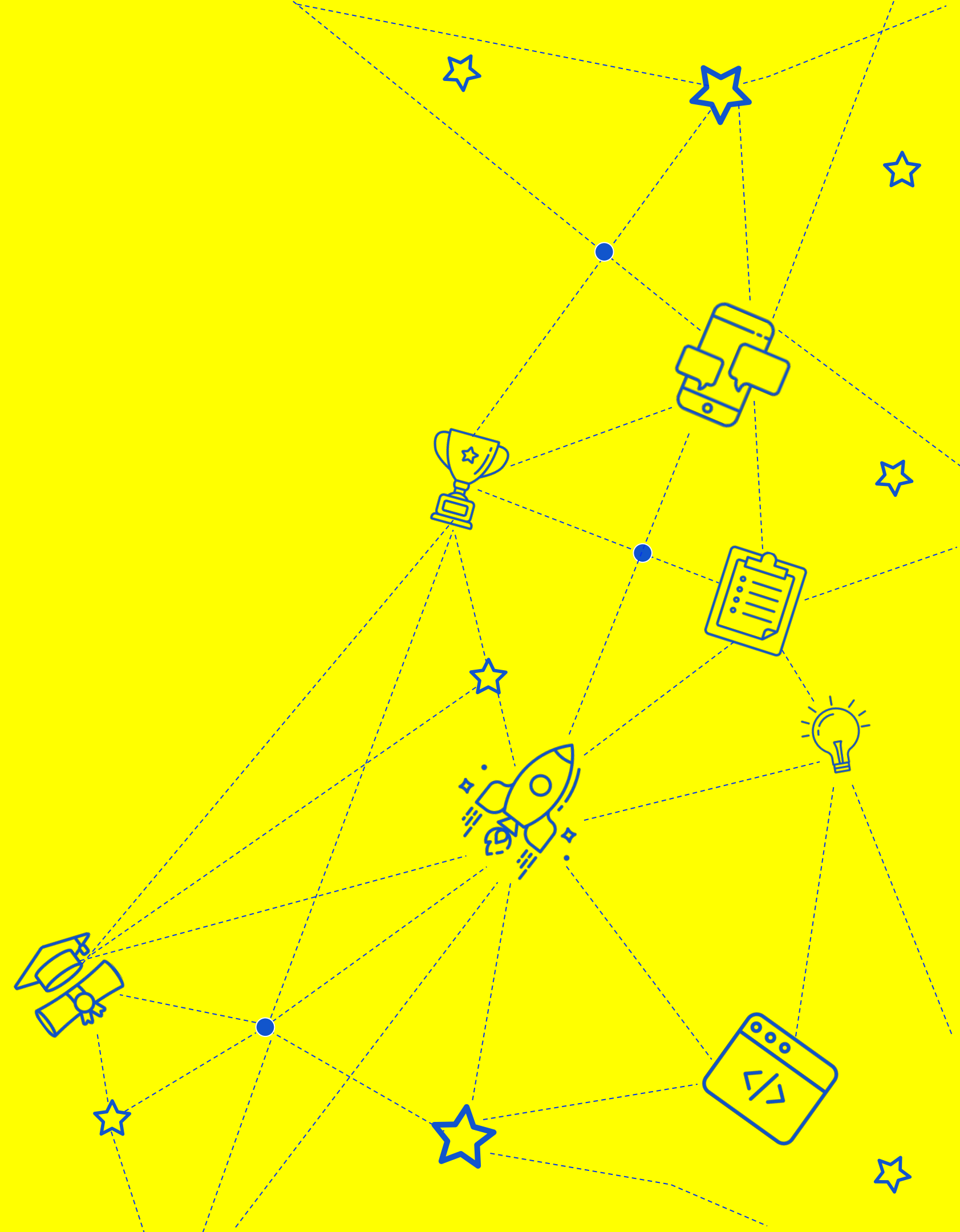
Mentor, Solvery.io

- Построил большой сервис из маленького стартапа
- Более 7 лет опыта в разработке на python
- Более 60 занятий с учениками на <https://solvery.io/ru/mentor/EvgrDan>

Содержание урока

- ★ Основные понятия
- ★ Типы исключений
- ★ Обработка исключений
- ★ Вызов исключений
- ★ Антипаттерны обработки исключений

Основные понятия



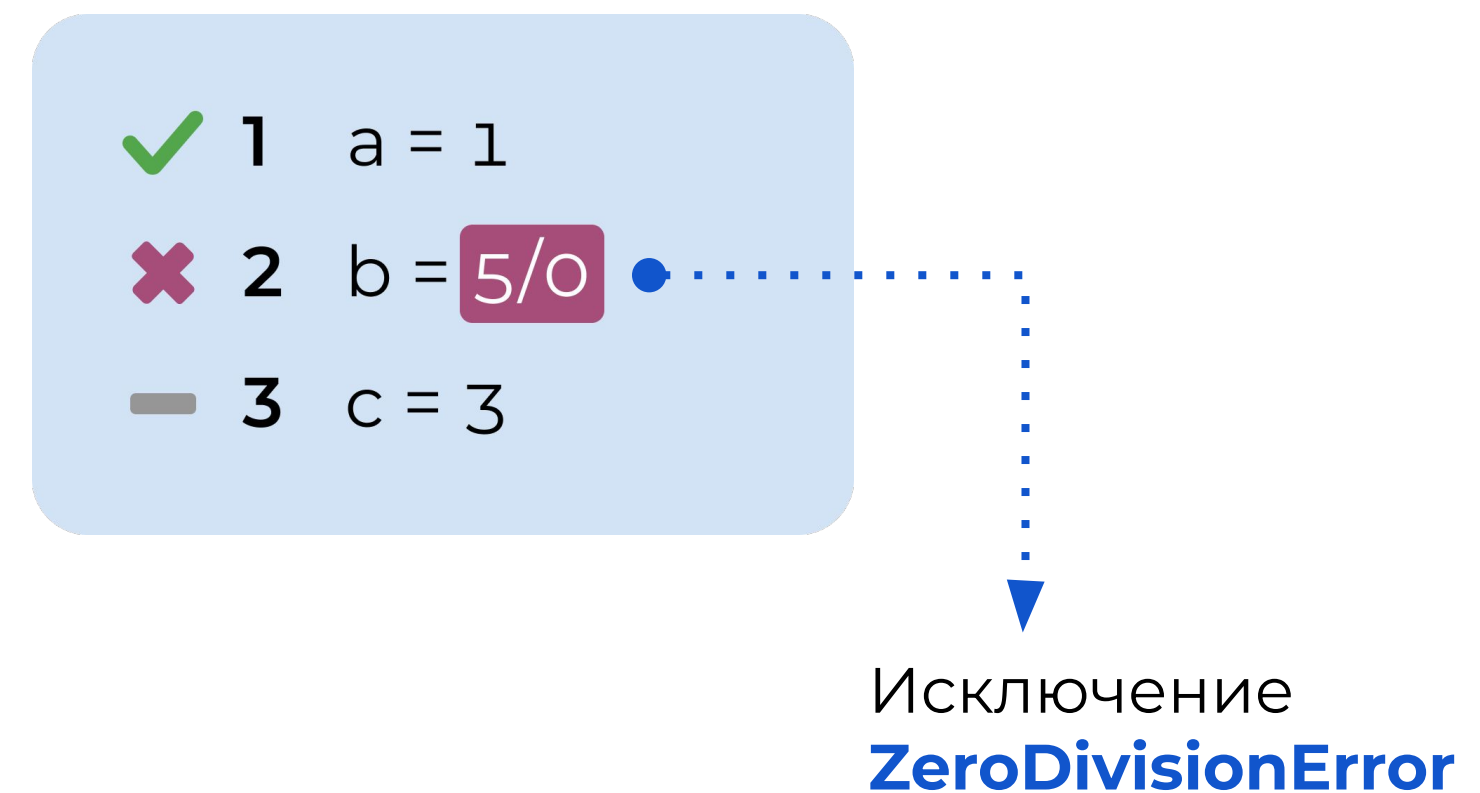
Исключение –

это событие, которое возникает в случае **ошибки**
или нетипичного поведения программы



ERROR.

Принцип работы исключений



Traceback



Показывает “путь” к месту ошибки через все вызываемые функции

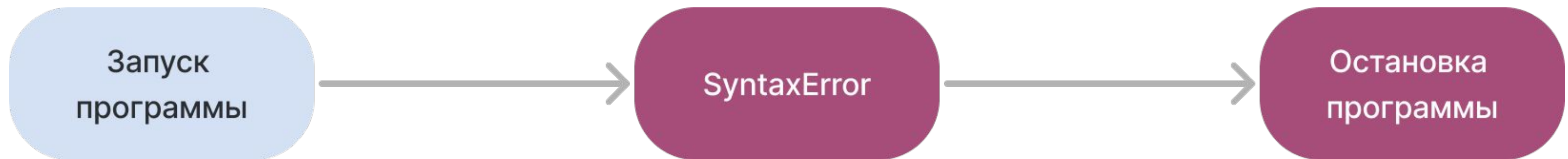
```
[12]: def divide(a, b):  
      return a / b
```

```
[14]: c = divide(5, 0)
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
Cell In[14], line 1  
----> 1 c = divide(5, 0)  
  
Cell In[12], line 2, in divide(a, b)  
      1 def divide(a, b):  
----> 2     return a / b  
  
ZeroDivisionError: division by zero
```

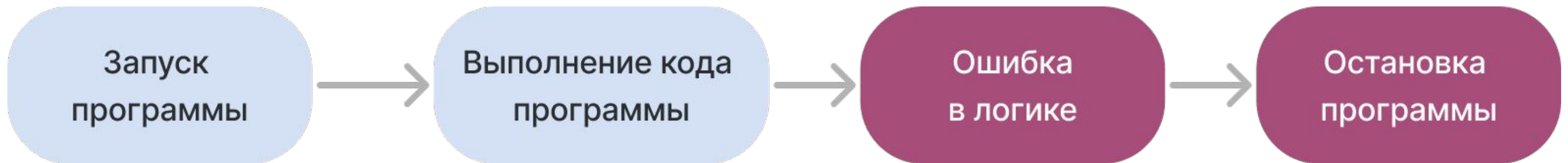
Причины возникновения

1 Ошибка в синтаксисе



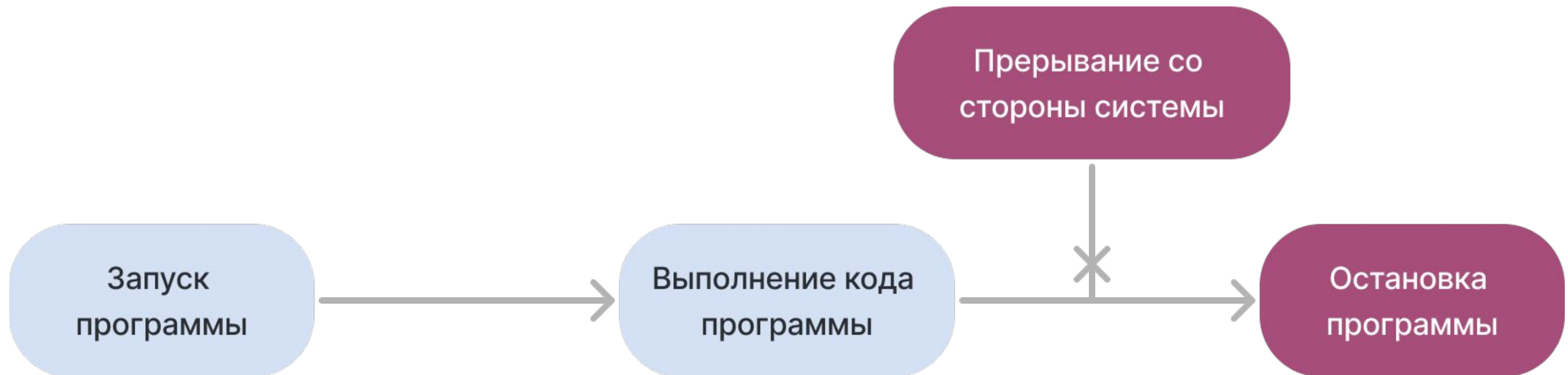
Причины возникновения

2 Ошибка в логике работы программы



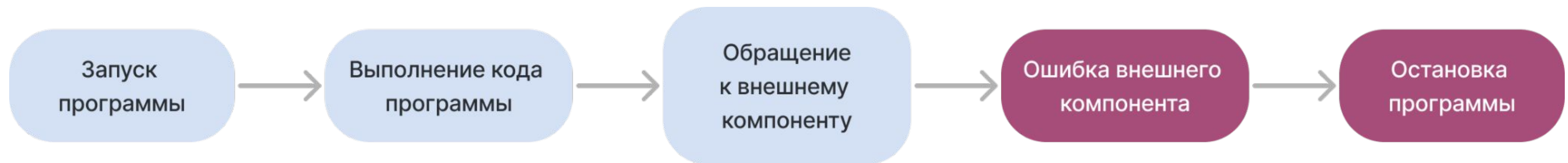
Причины возникновения

3 Системная ошибка



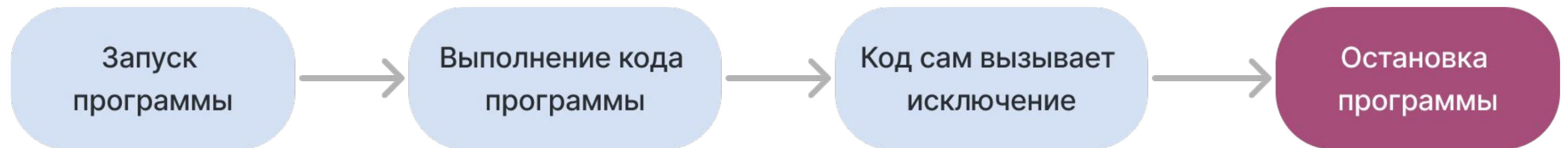
Причины возникновения

4 Ошибка в работе внешних компонентов

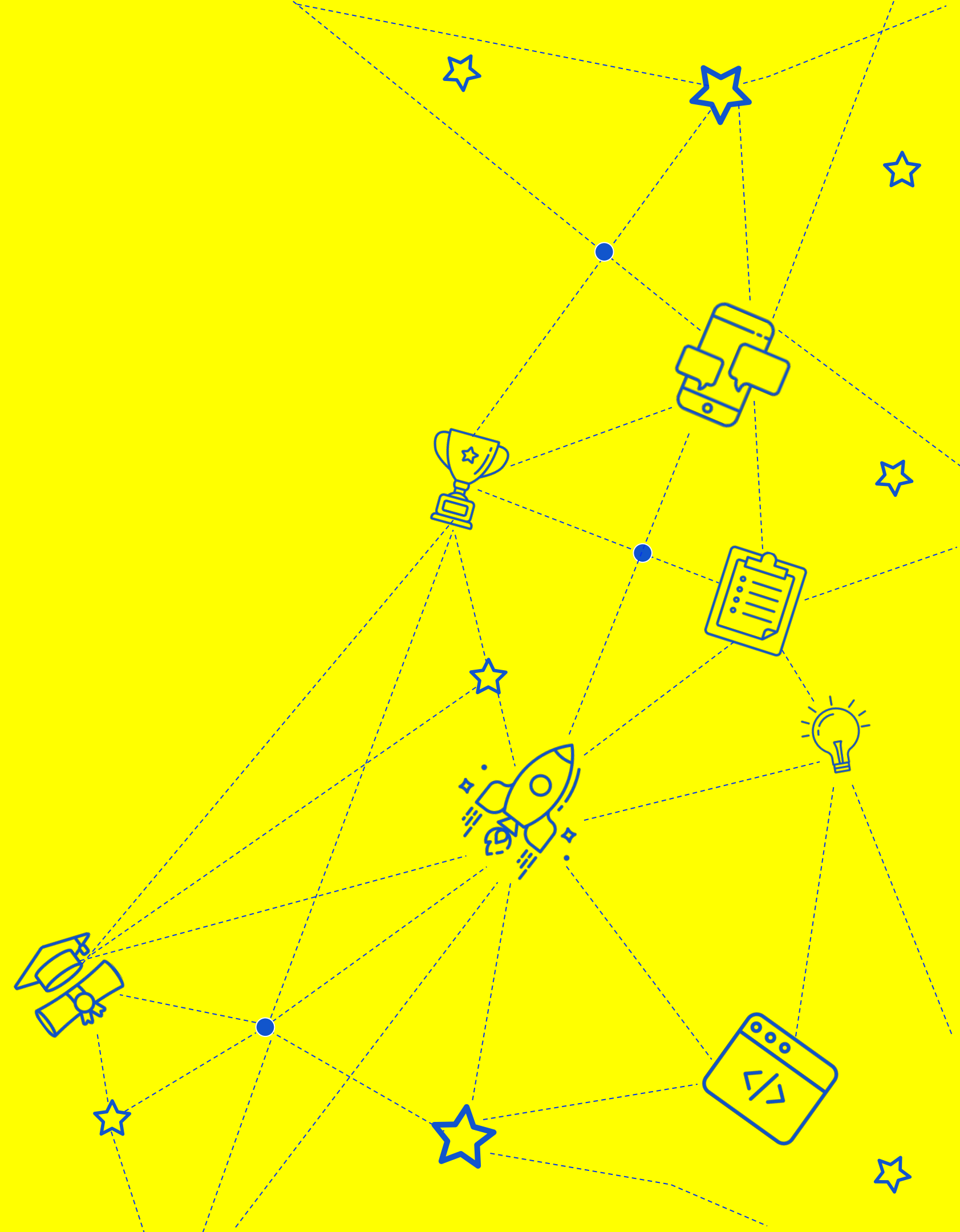


Причины возникновения

5 ЯВНЫЙ ВЫЗОВ ИСКЛЮЧЕНИЯ

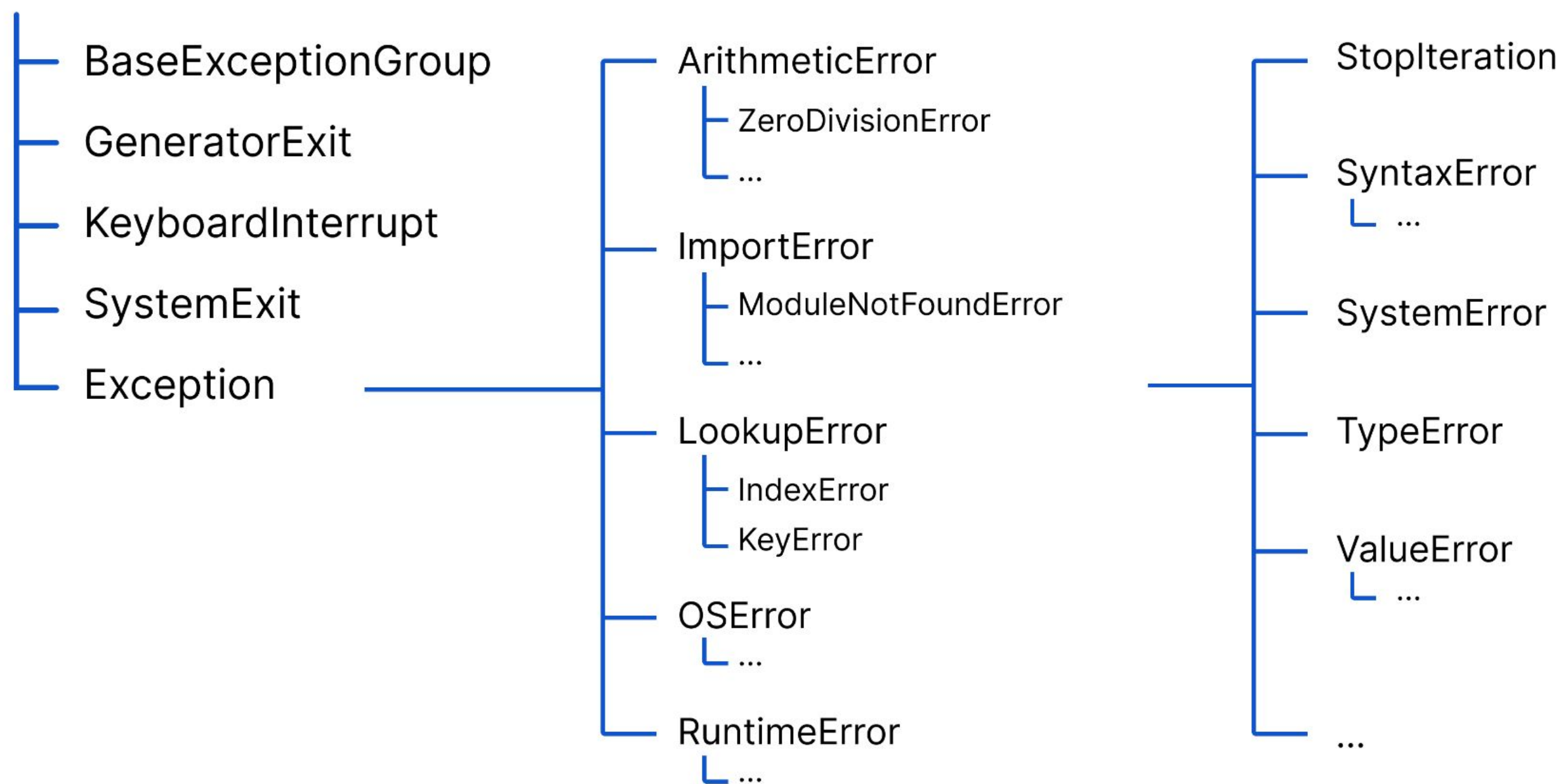


Типы исключений



Классы исключений

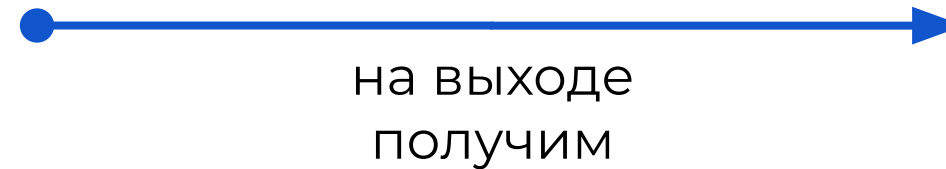
BaseException



Базовый класс исключений

BaseException - базовый класс

При передачи объекта исключения
в функцию **str**



Текстовое описание
ошибки

Атрибуты:

args

- кортеж аргументов, переданных
в класс при инициализации

Методы:

with_traceback(tb)

- позволяет задать traceback
для исключения

Стандартные исключения

SyntaxError



```
iff a == b:  
    print("Они равны!")
```

```
Cell In[22], line 1
```

```
    iff a == b:
```

```
        ^
```

```
SyntaxError: invalid syntax
```


Стандартные исключения

SyntaxError - как предотвратить

```
13  iff a == b:  
14      print("Они равны!")
```



Стандартные исключения

TypeError - несоответствие типов



```
a = 2  
b = "2"  
c = a + b
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[21], line 3  
      1 a = 2  
      2 b = "2"  
----> 3 c = a + b  
  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Стандартные исключения

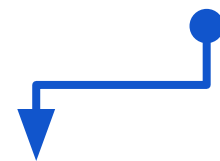
TypeError - как предотвратить



```
# Пример предотвращения TypeError  
a = 2  
b = int("2")  
c = a + b
```

Стандартные исключения

ValueError - неверное значение аргумента



```
number = "abc"  
result = int(number)  
print(result)
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[23], line 2  
      1 number = "abc"  
----> 2 result = int(number)  
      3 print(result)  
  
ValueError: invalid literal for int() with base 10: 'abc'
```



```
number = "123"  
result = int(number)  
print(result)
```

```
123
```

Стандартные исключения

ValueError - как предотвратить

Обращать внимание
на подсказки IDE



Явная типизация
(с python 3.5)



Стандартные исключения

IndexError - индекс вне диапазона элементов



```
my_list = ["a", "b"]  
print(my_list[2])
```

IndexError

Cell In[24], line 2

```
1 my_list = ["a", "b"]  
----> 2 print(my_list[2])
```

IndexError: list index out of range

Стандартные исключения

IndexError - как предотвратить

```
# Предотвращение IndexError  
my_list = ["a", "b"]  
if len(my_list) > 2:  
    print(my_list[2])
```



Стандартные исключения

KeyError - ключ не найден в словаре



```
person = {"name": "Marina", "age": 24}
print(person["last_name"])
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[25], line 2
      1 person = {"name": "Marina", "age": 24}
----> 2 print(person["last_name"])
```

KeyError: 'last_name'

Стандартные исключения

KeyError - как предотвратить



```
# Предотвращение KeyError  
person = {"name": "Marina", "age": 24}  
print(person.get("last_name", ""))
```

```
# Предотвращение KeyError 2  
person = {"name": "Marina", "age": 24}  
if "last_name" in person.keys():  
    print(person["last_name"])
```

Стандартные исключения

ZeroDivisionError - попытка деления на ноль



```
a = 5 / 0
```

```
-----  
ZeroDivisionError
```

```
Cell In[26], line 1
```

```
----> 1 a = 5 / 0
```

```
ZeroDivisionError: division by zero
```

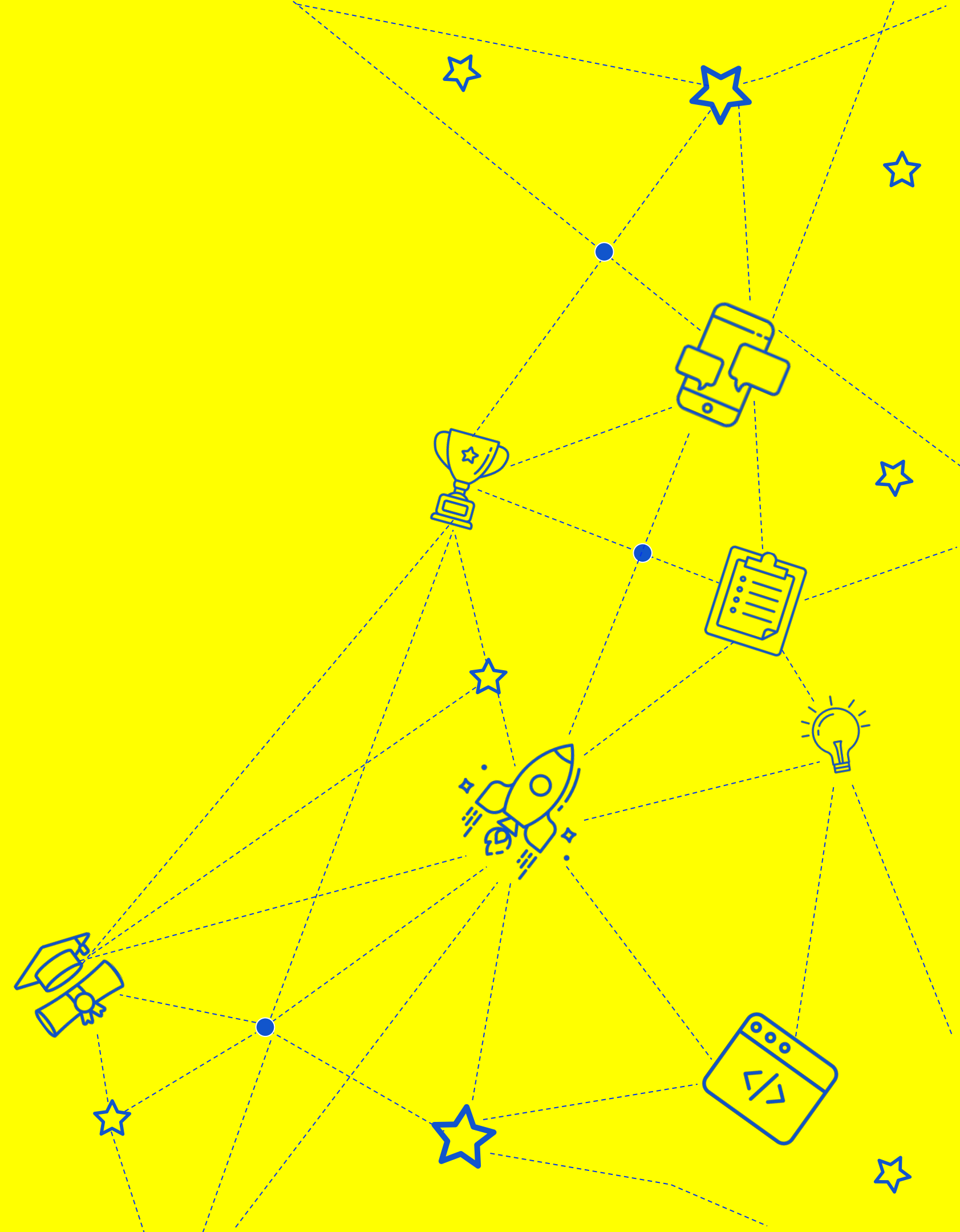
Стандартные исключения

ZeroDivisionError - как предотвратить

```
# Предотвращение ZeroDivisionError
x = 5
y = 0
if y != 0:
    a = x / y
else:
    a = 0
```

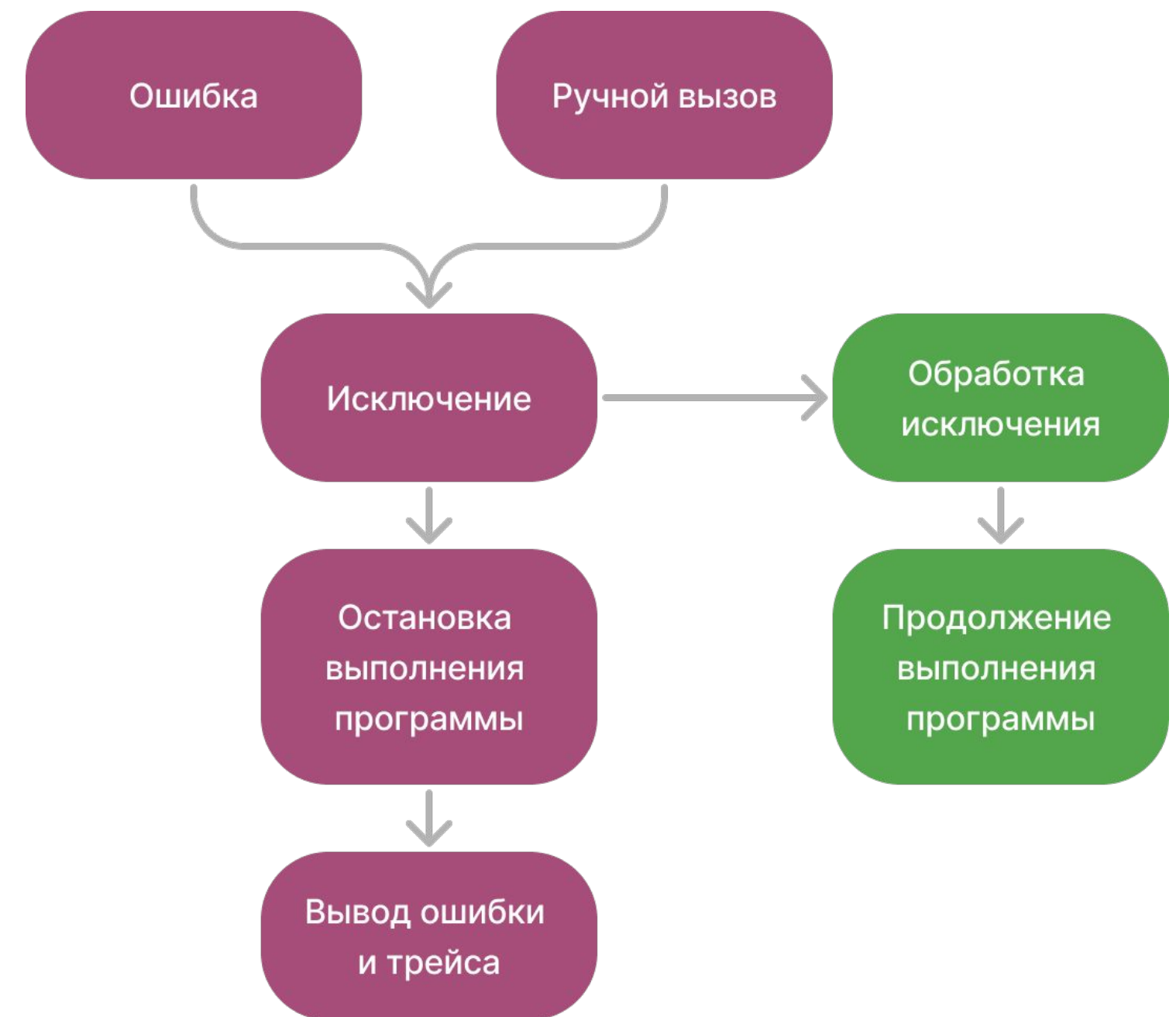


Обработка исключений



Обработка исключений

Процесс, при котором происходит “отлов” исключения для предотвращения завершения работы программы

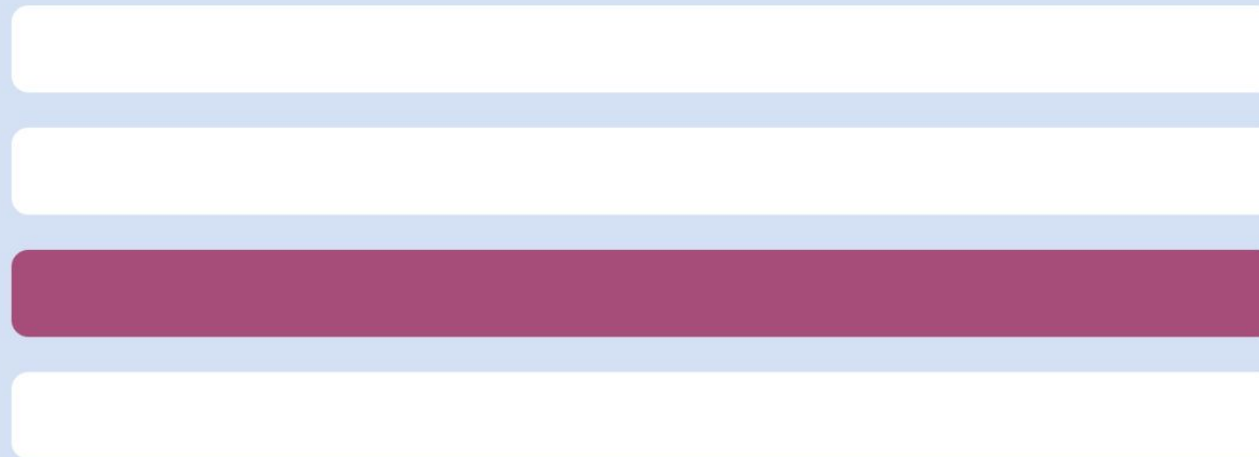


Для чего обрабатывать исключения

- ★ Для повышения отказоустойчивости программы
- ★ Для дополнения информации об ошибке
- ★ Для применения дополнительных мер в случае ошибки

Блоки try, except

try:



except:



Блоки try, except

```
try:  
    # код, который может вызвать исключения  
except ExceptionType:  
    # код, который обрабатывает исключение  
except AnotherException as ex:  
    # код, который обрабатывает другой класс исключения  
    print(ex)
```


Блок `finally`

```
try:  
    # код, который может вызвать исключения  
except ExceptionType:  
    # код, который обрабатывает исключение  
finally:  
    # код, который выполнится в любом случае
```

Блок else

```
try:  
    # код, который может вызвать исключение  
except ExceptionType:  
    # код, который обрабатывает исключение  
else:  
    # код, который выполнится только когда нет исключений
```

Вывод информации об ошибке

```
try:  
    a = 5/0  
except ZeroDivisionError as ex:  
    print(ex)
```

```
division by zero
```

Информация об исключении

Модуль `sys`:

- 1. Метод `exception()`

работает только начиная с версии python 3.11

- 2. Метод `exc_info()` - вернёт tuple:

- ★ Тип исключения
- ★ Объект исключения
- ★ Traceback исключения (`ex.__traceback__`)

Вывод номера строки

```
# Вывод номера строки  
import sys # Импорт модуля sys  
  
try:  
    a = 5 / 0  
except ZeroDivisionError as ex:  
    line_num = sys.exc_info()[-1].tb_lineno  
    print("Error in line %s: %s" % (str(line_num), str(ex)))
```

```
Error in line 5: division by zero
```


Вывод traceback

```
import sys
import traceback
try:
    a = 5/0
except ZeroDivisionError as ex:
    # получаем системную информацию об исключении
    exc_type, exc_value, exc_traceback = sys.exc_info()
    # форматируем информацию с помощью модуля traceback
    formatted = traceback.format_exception(exc_type, exc_value, exc_traceback)
    # Выводим на экран все строки форматированной информации
    print("Error: %s\r\n %s" % (str(ex), '\r\n'.join(formatted)))
```

Error: division by zero

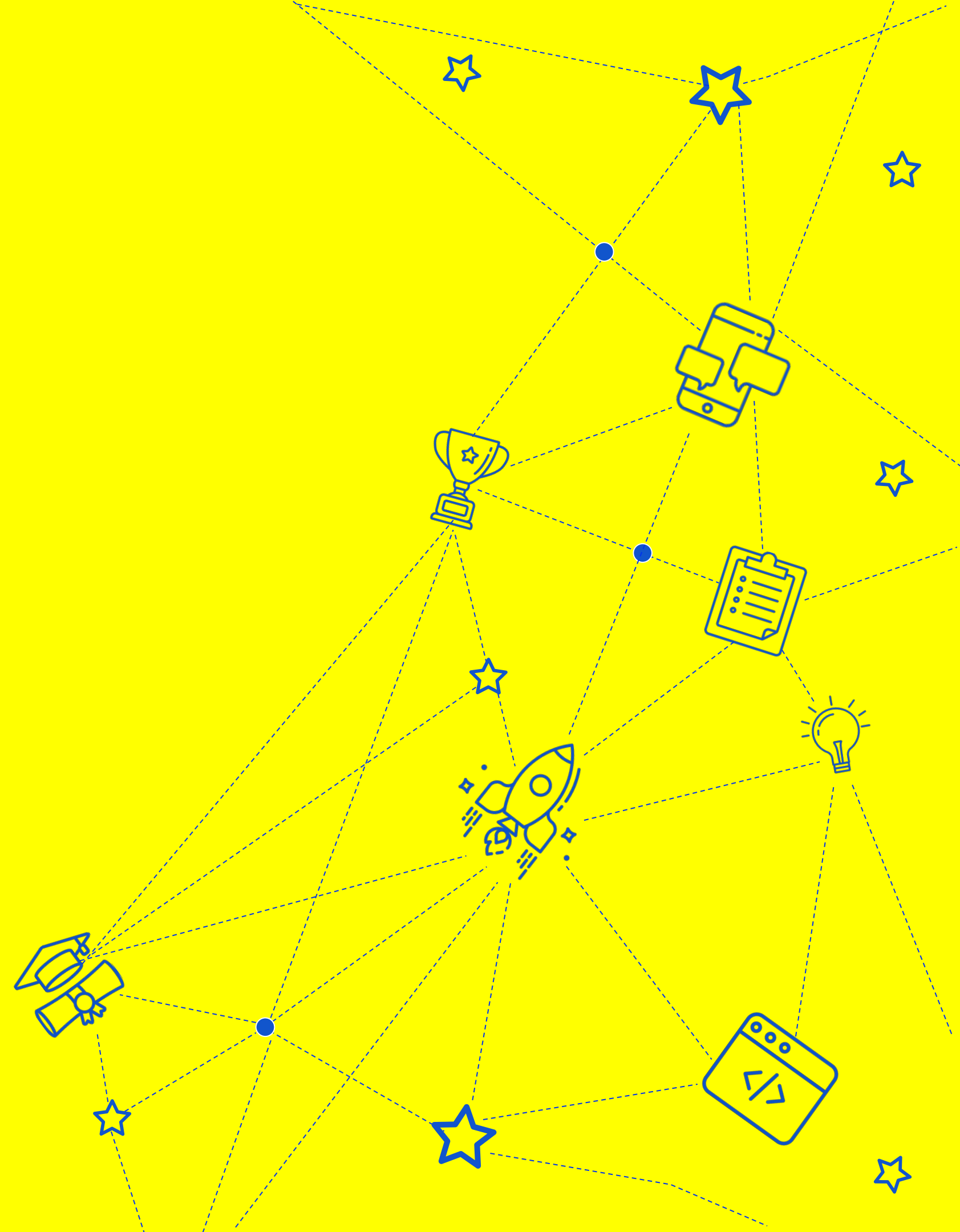
Traceback (most recent call last):

File "<ipython-input-34-cd75c72faf9d>", line 4, in <module>

a = 5/0

ZeroDivisionError: division by zero

Вызов и исключений



Вызов исключения –

инициализация события исключения самим разработчиком

Пользовательские исключения –

исключения, которые определены самим разработчиком

ВЫЗОВ ИСКЛЮЧЕНИЯ

Исключения могут быть вызваны самой программой с помощью функции **raise**

```
raise Exception("I called it")
```

```
-----  
Exception
```

```
Cell In[27], line 1
```

```
----> 1 raise Exception("I called it")
```

```
Exception: I called it
```

Область применения

Вызов исключений может использоваться в нескольких случаях:

Программа не смогла обработать ошибку самостоятельно



Неожиданные или неверные входные данные



При обработке исключения требуется отправить его выше по цепочке



Область применения

- Программа не смогла обработать ошибку самостоятельно

```
def divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError as ex:  
        if a == 0:  
            return 0  
        else:  
            raise ex
```

```
# Вернём 0  
print(divide(0, 0))  
# Выбросит исключение  
print(divide(5, 0))
```

Область применения

- Неожиданные или неверные входные данные

```
# Применение вызова исключения 2
def print_age(age):
    if age < 0:
        raise ValueError("Возраст не может быть меньше 0")
    else:
        print(age)

# Выведет 12
print_age(12)
# Выбросит исключение
print_age(-2)
```

Область применения

- Отправка исключения выше по цепочке

```
# Применение вызова исключения 3
def print_last_name(person):
    try:
        print(person["last_name"])
    except KeyError as ex:
        print(ex)
        raise ex

person = {"name": "Marina", "age": 24}
print_last_name(person)
```


Пользовательские исключения

- Исключения, определяемые самим разработчиком

```
class TooLongNameException(Exception):  
    pass
```

```
raise TooLongNameException("Слишком длинное имя")
```

```
-----  
TooLongNameException                                Traceback (most recent call last)
```

```
Cell In[28], line 4
```

```
      1 class TooLongNameException(Exception):  
      2     pass  
----> 4 raise TooLongNameException("Слишком длинное имя")
```

```
TooLongNameException: Слишком длинное имя
```

Область применения

Пользовательские исключения применяются в нескольких случаях:

- 1 Разделение обработки на разные блоки
- 2 Стандартные не подходят
- 3 Уточнение сообщения

Область применения

- Разделение обработки на разные блоки

```
# Разделение обработки на разные блоки
class TooLongNameException(Exception):
    pass

class TooShortNameException(Exception):
    pass

def check_name(name):
    if len(name) > 10:
        raise TooLongNameException("Слишком длинное имя")
    if len(name) < 2:
        raise TooShortNameException("Слишком короткое имя")
    return True
```


Область применения

- Стандартные исключения не подходят

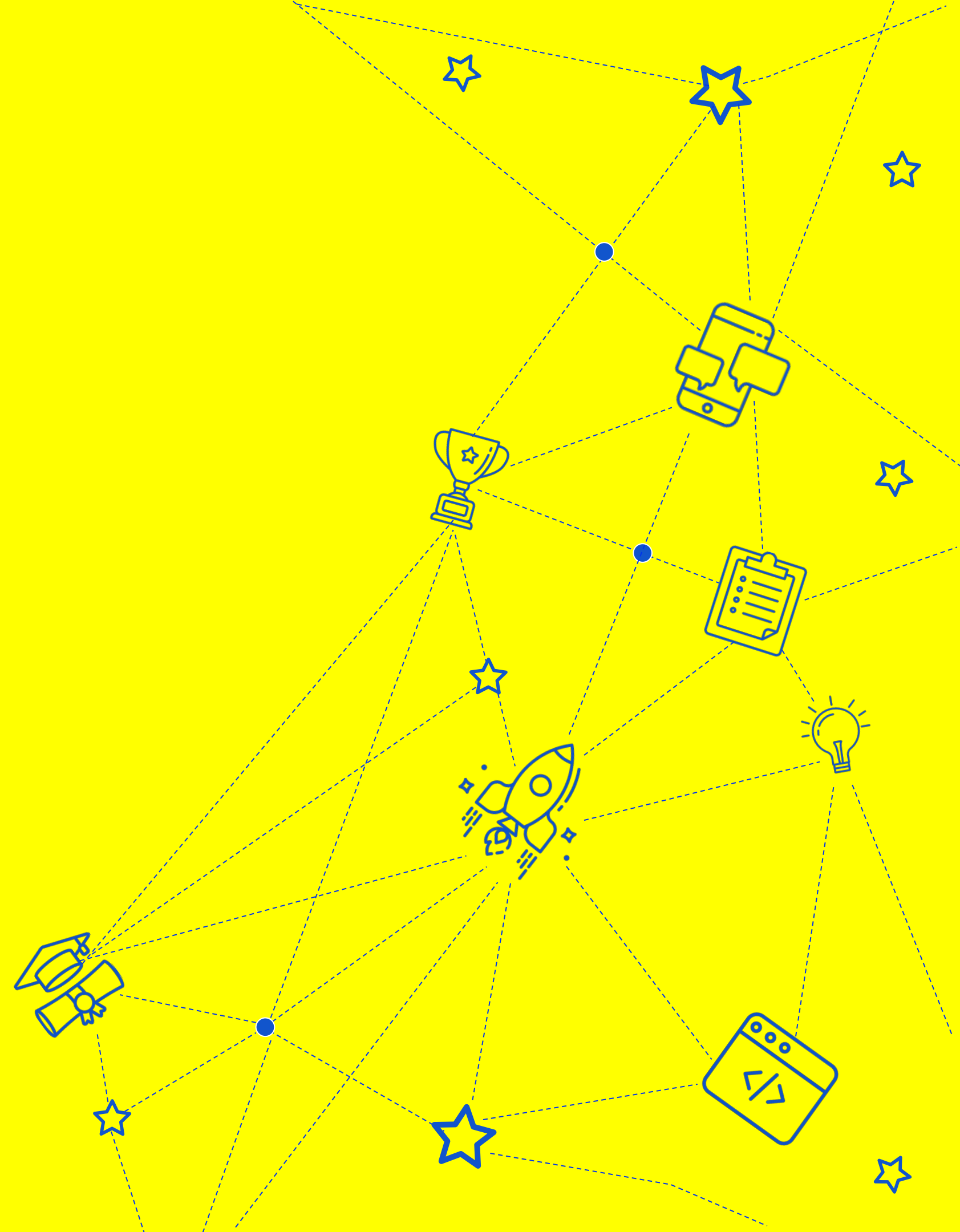
```
class NameIsEmpty(Exception):  
    pass  
  
def check_name(name):  
    if name == "":  
        raise NameIsEmpty("Имя не может быть пустым")
```

Область применения

- Уточнение сообщения

```
class LastNameNotFound(Exception):  
    pass  
  
class NameNotFound(Exception):  
    pass  
  
def get_name_and_last_name(person):  
    try:  
        return person["name"], person["last_name"]  
    except KeyError:  
        if "name" in person.keys():  
            raise LastNameNotFound("Фамилия не найдена")  
        else:  
            raise NameNotFound("Имя не найдено")
```

Антипаттерны обработки исключений



pass & try-except

```
try:  
    a = 5/0  
except ZeroDivisionError as ex:  
    pass
```

except без указания типа

```
try:  
    a = 5/0  
except Exception as ex:  
    print(ex)
```


raise базового исключения

```
# Пример raise базового исключения
def get_last_name(person):
    # Проверяем наличие ключа в словаре person
    if "last_name" not in person.keys():
        raise Exception("Ключ не найден")
    return person["last_name"]

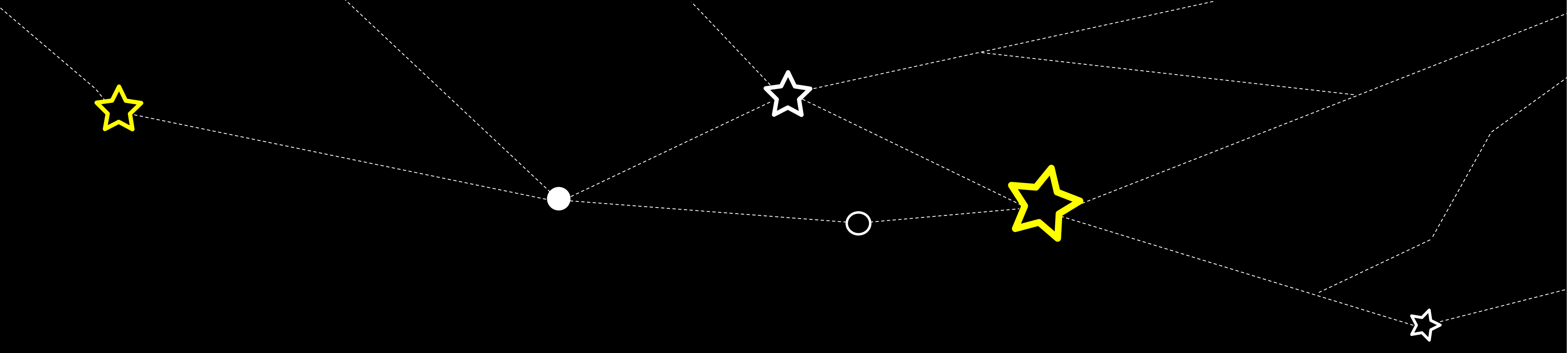
person = {"name": "Marina", "age": 24}
try:
    print(get_last_name(person))
except KeyError:
    print("Фамилия не найдена")
```


Исключения как часть логики

```
# Пример исключений как части логики
person = {"name": "Marina", "age": 24}
try:
    last_name = person["last_name"]
except KeyError:
    last_name = ""
print(last_name)
```

Много кода внутри одного try

```
# Много кода внутри одного try  
person = {"name": "Marina", "age": 24}  
try:  
    print(person["name"])  
    print(person["last_name"])  
except KeyError:  
    print("Фамилия не найдена")
```

СПАСИБО ЗА ВНИМАНИЕ



<https://vk.com/evgrdan>



@evgrdan



@EvgrDan



<https://solvery.io/ru/mentor/EvgrDan>